

MULTI USER LOGIN SYSTEM

A PROJECT REPORT

Submitted by

AARON VARUGHESE BINU [Reg No: RA2211030010171]

JOEL JACOB [Reg No: RA2211030010182]

Under the Guidance of

DR. S. RAMESH

Assistant Professor,

Department of Networking and Communications

*In partial fulfilment of the requirements for the degree of **BACHELOR OF TECHNOLOGY in COMPUTER SCIENCE AND ENGINEERING**
with a specialization in **CYBER SECURITY***



**DEPARTMENT OF NETWORKING AND COMMUNICATION
COLLEGE OF ENGINEERING AND TECHNOLOGY, SRM
INSTITUTE OF SCIENCE AND TECHNOLOGY,
KATTANKULATHUR – 603203**

NOVEMBER 2023



SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

KATTANKULATHUR – 603 203

BONAFIDE CERTIFICATE

Certified that this B.Tech project report titled “**MULTI USER LOGIN SYSTEM**” is the bonafide work of Aaron Varughese Binu [Reg. No:RA2211030010171] and Joel Jacob [Reg.No:RA2211030010182] who carried out the project work under my supervision. Certified further, that to the best of my knowledge the work reported herein does not form part of any other thesis or dissertation on the basis of which a degree or award was conferred on an earlier occasion for this or any other candidate.

FACULTY IN-CHARGE

Dr. S.Ramesh
Assistant Professor
Department of Networking and
Communications
SRM Institute of Science and
Technology
Kattankulathur

HEAD OF THE DEPARTMENT

Dr. Annapurani. K
Professor and Head
Department of Networking and
Communications
SRM Institute of Science and
Technology
Kattankulathur

Department of NETWORKING AND COMMUNICATION
SRM Institute of Science and Technology

Own Work Declaration Form

Degree/ Course: B.Tech in Computer Science and Engineering with a specialization in Cyber Security

Student Names: Aaron Varughese Binu, Joel Jacob

Registration Numbers: RA2211030010171, RA2211030010182

Title of Work: Multi User Login System

We hereby certify that this assessment compiles with the University's Rules and Regulations relating to Academic misconduct and plagiarism, as listed in the University Website, Regulations, and the Education Committee guidelines.

We confirm that all the work contained in this assessment is our own except where indicated, and that we have met the following conditions:

- Clearly references / listed all sources as appropriate
- Referenced and put in inverted commas all quoted text (from books, web, etc.)
- Given the sources of all pictures, data etc. that are not my own.
- Not made any use of the report(s) or essay(s) of any other student(s) either past or present.
- Acknowledged in appropriate places any help that I have received from others. (e.g. fellow students, technicians, statisticians, external sources) • Compiled with any other plagiarism criteria specified in the Course handbook / University website I understand that any false claim for this work will be penalized in accordance with the University policies and regulations.

ACKNOWLEDGEMENT

We express our humble gratitude to **Dr. C. Muthamizhchelvan**, Vice-Chancellor, SRM Institute of Science and Technology, for the facilities extended for the project work and his continued support.

We extend our sincere thanks to Dean-CET, SRM Institute of Science and Technology, **Dr. T.V.Gopal**, for his invaluable support.

We wish to thank **Dr. Revathi Venkataraman**, Professor & Chairperson, School of Computing, SRM Institute of Science and Technology, for her support throughout the project work.

We are incredibly grateful to our Head of the Department, **Dr. Annapurani Panaiyappan K**, Professor, Department of Networking and Communication, SRM Institute of Science and Technology, for her suggestions and encouragement at all the stages of the project work.

We register our immeasurable thanks to our Faculty Advisor, **Dr. M. SHOBANA**, Assistant Professor, Department of NETWORKING AND COMMUNICATION, SRM Institute of Science and Technology, for leading and helping us to complete our course.

Our inexpressible respect and thanks to our guide, **Dr. S RAMESH**, Assistant Professor, Department of NETWORKING AND COMMUNICATION, SRM Institute of Science and Technology, for providing us with an opportunity to pursue our Report under his mentorship. He provided us with the freedom and support to explore the research topics of our interest. His passion for solving problems and making a difference in the world has always been inspiring.

We sincerely thank the NETWORKING AND COMMUNICATION Department staff and students, SRM Institute of Science and Technology, for their help during our Report. Finally, we would like to thank parents, family members, and friends for their unconditional love, constant support, and encouragement.

Aaron Varughese Binu [RA2211030010171]

Joel Jacob [RA2211030010182]

TABLE OF CONTENTS

S.NO	TITLE	PAGE NO.
1	ABSTRACT	7
2	INTRODUCTION	8
3	LITERATURE SURVEY	13
4	PROPOSED METHODOLOGY	14
5	SOURCE CODE	15
6	RESULTS	19
7	CONCLUSION	21

ABSTRACT

The Multi-User Login System project is designed to provide a secure and efficient authentication mechanism for multiple users accessing a shared system. In today's digital age, ensuring the privacy and security of user data is of utmost importance. This project addresses the need for a robust login system that supports concurrent access by multiple users while incorporating strong password policies.

The system employs a multi-threaded approach to handle user registration and authentication concurrently, enhancing the overall efficiency and user experience. Each user is required to register with a unique username and a strong password, meeting specific criteria to ensure security. The registration process includes checking for the existence of duplicate usernames and validating password strength.

INTRODUCTION

1.1. General

In the realm of computer systems and applications, the concept of a multi-user login system plays a pivotal role in facilitating secure and efficient access to shared resources. This type of system allows multiple users to interact with a computer, network, or application concurrently, each with their unique set of credentials and permissions. The fundamental purpose of a multi-user login system is to manage user identities, control access to resources, and ensure the security and integrity of sensitive information.

Key Components of Multi-User Login Systems:

User Identification: Users are uniquely identified within the system through credentials such as usernames and passwords. This identification is crucial for personalizing user experiences and enforcing security measures.

Authentication Mechanism: The authentication process verifies the identity of users attempting to access the system. This often involves the use of passwords, biometrics, or two-factor authentication methods to enhance security.

Authorization and Permissions: Once authenticated, users are granted specific permissions or access levels based on their roles or privileges. Authorization mechanisms control what actions users can perform

1.2. Purpose

The purpose of a Multi-User Login System is to provide a secure and organized way for multiple individuals to access and interact with a shared computer system, network, or application. This system serves several essential purposes:

User Identification and Authentication: Establishes a unique identity for each user through usernames and passwords or other authentication methods. Verifies the authenticity of users before granting access to the system.

Concurrency Management: Handles simultaneous access by multiple users, ensuring that their interactions do not interfere with each other. Utilizes multi-threading or other concurrency management techniques to enhance system responsiveness.

Security Enhancement: Enforces strong password policies and employs encryption to protect user credentials and sensitive information. Implements security measures to safeguard against unauthorized access and potential security threats.

User Accountability and Auditing: Maintains logs of user activities for auditing purposes, allowing administrators to track and review user interactions with the system. Enhances accountability by attributing actions to specific user identities.

User Management: Facilitates the creation, modification, and deletion of user accounts by system administrators. Streamlines the process of managing user profiles, ensuring the system remains organized and up-to-date.

User-Friendly Interfaces: Provides user-friendly interfaces for login, navigation, and interaction with the system. Ensures a positive user experience by presenting information in a clear and intuitive manner.

1.3. Scope

The scope of a Multi-User Login System project encompasses a broad range of functionalities and considerations that contribute to the successful implementation and deployment of a secure and efficient system. The scope outlines the boundaries and objectives of the project, defining what features and aspects will be included. Here are key components within the scope of a Multi-User Login System project:

User Registration and Authentication: Implement mechanisms for user registration, ensuring unique usernames and strong password requirements. Develop a secure authentication process to validate user credentials.

Concurrency Management: Implement multi-threading or concurrency mechanisms to handle simultaneous user interactions. Ensure that concurrent access does not compromise data integrity or system stability.

Security Measures: Enforce strong password policies and encryption techniques to protect user credentials. Implement secure storage mechanisms for user data to prevent unauthorized access.

Logging and Auditing: Develop a comprehensive logging system to record user activities. Implement auditing features to monitor and review system interactions for security and compliance purposes.

User Management: Create, update, and delete user accounts securely. Provide administrative tools for managing user profiles and permissions.

User-Friendly Interfaces: Design intuitive and user-friendly interfaces for login, navigation, and interaction. Prioritize a positive user experience with clear and easily understandable interfaces.

Exit Mechanism: Design a smooth and secure exit mechanism for users to log out of the system. Ensure proper clean up and closure of user sessions.

1.4 PROBLEM DESCRIPTION

The Multi-User Login System seeks to address key challenges associated with user authentication and management in shared computing environments. Security concerns, including vulnerabilities in traditional authentication methods, necessitate the implementation of robust measures to protect user credentials. Concurrency management is critical to handling simultaneous user interactions without compromising system stability. Ensuring precise access control and authorization mechanisms is essential to protect sensitive data. The complexity of user management, scalability requirements, and the need for error handling and recovery pose additional challenges. The system aims to overcome these issues by providing a secure, efficient, and user-friendly solution that adheres to regulations, enables seamless collaboration, and offers scalability for evolving user needs.

1.5 The Need for a Multi-Thread

Simultaneous User Actions: In a multi-user system, several users may attempt to register, log in, or perform other actions simultaneously. Multi-threading enables the system to handle these actions concurrently, preventing delays and ensuring a smooth user experience.

Concurrency Management: Multi-threading facilitates the effective management of concurrent user sessions. Without multi-threading, the system might be forced to process tasks sequentially, leading to potential bottlenecks and decreased responsiveness.

Improved Responsiveness: By allowing multiple tasks to execute concurrently, the system becomes more responsive. Users experience quicker response times, reducing the perceived latency during interactions such as registration, login, or other operations.

Optimal Resource Utilization: Multi-threading optimizes the use of system resources, ensuring that CPU cycles and memory are efficiently distributed among

concurrent tasks. This results in better utilization of available resources and improved overall system performance.

Enhanced Throughput: Multi-threading enhances system throughput by allowing multiple operations to be processed in parallel. This is particularly crucial in scenarios where a large number of users may be accessing the system simultaneously.

1.6 OBJECTIVE OF THE RESULT

Performance Enhancement: Improve system performance based on scalability and concurrency testing results to ensure optimal responsiveness under varying user loads.

Security Strengthening: Address identified security vulnerabilities and enhance the system's security measures to safeguard user credentials and sensitive data

Usability and User Experience Improvement: Enhance the user interface and overall user experience based on feedback and usability testing results.

Compliance Assurance: Ensure continued compliance with data protection and privacy regulations based on the results of compliance checks.

User Management Streamlining: Optimize user management functionalities based on testing results, including account creation, updates, and deletions.

LITERATURE SURVEY

The literature survey of a Multi-User Login System delves into an extensive exploration of existing research and publications related to user authentication, access control, concurrency management, security measures, and overall system functionality. Studies on user authentication methods, including password-based, biometric, and multi-factor approaches, provide insights into securing user credentials. Concurrently, research on multi-threading and concurrency management techniques sheds light on optimizing system performance and responsiveness under varying user loads. Security considerations in user management systems, encompassing encryption and secure storage, are explored to safeguard sensitive information. Additionally, the survey investigates the significance of user experience and usability principles in the design of secure login systems. Scalability, error handling strategies, compliance with regulations, collaborative features, and case studies contribute to a holistic understanding of multi-user login systems. The literature survey also explores future trends and emerging technologies, guiding researchers and developers toward innovative approaches for creating secure, scalable, and user-centric login systems.

PROPOSED METHODOLOGY

Requirement Analysis: Conduct a comprehensive analysis of system requirements, considering factors such as user authentication, access control, security measures, concurrency management, scalability, and user experience.

Literature Review: Review existing literature on multi-user login systems, with a specific focus on the implementation of multi-threading for concurrent user interactions. Identify best practices, challenges, and solutions.

System Design: Design a robust system architecture that integrates multi-threading to handle concurrent user actions. Define modules for user authentication, access control, user management, and other relevant functionalities.

User Authentication and Access Control: Implement secure user authentication mechanisms, considering multi-factor authentication for enhanced security. Develop access control mechanisms based on user roles and permissions.

Concurrency Management: Integrate multi-threading techniques to manage concurrent user sessions efficiently. Implement thread-safe data structures and synchronization mechanisms to prevent data conflicts.

User Experience Enhancement: Design an intuitive and user-friendly interface, considering feedback from usability testing. Prioritize a positive user experience during login, navigation, and interaction.

Testing and Validation: Conduct thorough testing of each module and the system as a whole. Perform unit testing, integration testing, and system testing to validate the functionality, security, and performance of the multi-user login system.

SOURCE CODE

```
main.c
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  #include <pthread.h>
5  #include <ctype.h>
6
7  #define MAX_USERS 10
8  #define MAX_USERNAME_LENGTH 20
9  #define MAX_PASSWORD_LENGTH 20
10
11  // Structure to represent a user
12  typedef struct {
13      char username[MAX_USERNAME_LENGTH];
14      char password[MAX_PASSWORD_LENGTH];
15  } User;
16
17  // Database of users
18  User userDatabase[MAX_USERS];
19
20  // Mutex for thread synchronization
21  pthread_mutex_t mutex;
22
23  // Function to check the strength of the password
24  int isStrongPassword(const char* password) {
25      int length = strlen(password);
26      int hasUppercase = 0, hasLowercase = 0, hasDigit = 0, hasSpecialChar = 0;
```

```

27
28 for (int i = 0; i < length; ++i) {
29     if (isupper(password[i])) {
30         hasUppercase = 1;
31     } else if (islower(password[i])) {
32         hasLowercase = 1;
33     } else if (isdigit(password[i])) {
34         hasDigit = 1;
35     } else {
36         hasSpecialChar = 1;
37     }
38 }
39
40 // Password is considered strong if it meets all criteria
41 return (length >= 8) && hasUppercase && hasLowercase && hasDigit && hasSpecialChar;
42
43
44 // Function to register a new user
45 void* registerUser(void* arg) {
46     pthread_mutex_lock(&mutex);
47
48     printf("\nEnter a new username: ");
49     char newUsername[MAX_USERNAME_LENGTH];
50     scanf("%s", newUsername);
51
52     // Check if the username already exists
53     for (int i = 0; i < MAX_USERS; ++i) {
54         if (strcmp(userDatabase[i].username, newUsername) == 0) {
55             printf("Username already exists. Registration failed.\n");
56             pthread_mutex_unlock(&mutex);
57             pthread_exit(NULL);
58         }
59     }
60

```

```

60
61 // Find an empty slot in the database
62 int emptySlot = -1;
63 for (int i = 0; i < MAX_USERS; ++i) {
64     if (strlen(userDatabase[i].username) == 0) {
65         emptySlot = i;
66         break;
67     }
68 }
69
70 // Register the new user
71 strcpy(userDatabase[emptySlot].username, newUsername);
72
73 // Prompt the user to enter a strong password
74 char newPassword[MAX_PASSWORD_LENGTH];
75 do {
76     printf("Enter a strong password (at least 8 characters, including uppercase, lowercase, digit, and special character): ");
77     scanf("%s", newPassword);
78
79     if (!isStrongPassword(newPassword)) {
80         printf("Weak password. Please follow the password criteria.\n");
81     }
82 } while (!isStrongPassword(newPassword));
83
84 strcpy(userDatabase[emptySlot].password, newPassword);
85
86 printf("Registration successful for user %s.\n", newUsername);
87
88 pthread_mutex_unlock(&mutex);
89 pthread_exit(NULL);
90 }
91 }
92

```



```

93 // Function to authenticate a user
94 void* authenticateUser(void* arg) {
95     pthread_mutex_lock(&mutex);
96
97     printf("\nEnter your username: ");
98     char enteredUsername[MAX_USERNAME_LENGTH];
99     scanf("%s", enteredUsername);
100
101     // Find the user in the database
102     int userIndex = -1;
103     for (int i = 0; i < MAX_USERS; ++i) {
104         if (strcmp(userDatabase[i].username, enteredUsername) == 0) {
105             userIndex = i;
106             break;
107         }
108     }
109
110     // Check if the user was found
111     if (userIndex == -1 || strlen(userDatabase[userIndex].username) == 0) {
112         printf("User not found. Authentication failed.\n");
113         pthread_mutex_unlock(&mutex);
114         pthread_exit(NULL);
115     }
116
117     printf("Enter your password: ");
118     char enteredPassword[MAX_PASSWORD_LENGTH];
119     scanf("%s", enteredPassword);
120
121     // Check the entered password
122     if (strcmp(userDatabase[userIndex].password, enteredPassword) == 0) {
123         printf("Login successful! Welcome, %s.\n", enteredUsername);
124     } else {
125         printf("Login failed. Incorrect password.\n");
126     }

```

```

127     pthread_mutex_unlock(&mutex);
128     pthread_exit(NULL);
129 }
130
131 // Function to delete a user
132 void* deleteUser(void* arg) {
133     pthread_mutex_lock(&mutex);
134
135     printf("\nEnter the username to delete: ");
136     char deleteUserName[MAX_USERNAME_LENGTH];
137     scanf("%s", deleteUserName);
138
139     // Find the user in the database
140     int userIndex = -1;
141     for (int i = 0; i < MAX_USERS; ++i) {
142         if (strcmp(userDatabase[i].username, deleteUserName) == 0) {
143             userIndex = i;
144             break;
145         }
146     }
147
148     // Check if the user was found
149     if (userIndex == -1 || strlen(userDatabase[userIndex].username) == 0) {
150         printf("User not found. Deletion failed.\n");
151     } else {
152         // Delete the user
153         strcpy(userDatabase[userIndex].username, "");
154         strcpy(userDatabase[userIndex].password, "");
155         printf("User %s has been deleted.\n", deleteUserName);
156     }
157
158     pthread_mutex_unlock(&mutex);
159     pthread_exit(NULL);
160 }
161 }

```

```

162
163 // Function to display the menu
164 void displayMenu() {
165     printf("\nMenu:\n");
166     printf("1. Register\n");
167     printf("2. Authenticate\n");
168     printf("3. Delete User\n");
169     printf("4. Exit\n");
170 }
171
172 int main() {
173     // Initialize the user database
174     for (int i = 0; i < MAX_USERS; ++i) {
175         strcpy(userDatabase[i].username, "");
176         strcpy(userDatabase[i].password, "");
177     }
178
179     // Initialize the mutex
180     if (pthread_mutex_init(&mutex, NULL) != 0) {
181         printf("Mutex initialization failed.\n");
182         return 1;
183     }
184
185     int choice;
186     while (1) {
187         displayMenu();
188         printf("Enter your choice: ");
189         scanf("%d", &choice);

```

```

190
191     switch (choice) {
192     case 1:
193         pthread_t registerThread;
194         pthread_create(&registerThread, NULL, registerUser, NULL);
195         pthread_join(registerThread, NULL);
196         break;
197     case 2:
198         pthread_t authenticateThread;
199         pthread_create(&authenticateThread, NULL, authenticateUser, NULL);
200         pthread_join(authenticateThread, NULL);
201         break;
202     case 3:
203         pthread_t deleteThread;
204         pthread_create(&deleteThread, NULL, deleteUser, NULL);
205         pthread_join(deleteThread, NULL);
206         break;
207     case 4:
208         printf("Goodbye!\n");
209         break;
210     default:
211         printf("Invalid choice. Please enter a valid option.\n");
212     }
213
214     if (choice == 4) {
215         break;
216     }
217 }
218
219 // Destroy the mutex
220 pthread_mutex_destroy(&mutex);
221
222 return 0;
223 }

```

RESULTS

```
Menu:
1. Register
2. Authenticate
3. Delete User
4. Exit
Enter your choice: 1

Enter a new username: joel
Enter a strong password (at least 8 characters, including uppercase, lowercase, digit, and special character): joelj123
Weak password. Please follow the password criteria.
Enter a strong password (at least 8 characters, including uppercase, lowercase, digit, and special character): Joelj@123
Registration successful for user joel.

Menu:
1. Register
2. Authenticate
3. Delete User
4. Exit
Enter your choice: 1

Enter a new username: jj456
Enter a strong password (at least 8 characters, including uppercase, lowercase, digit, and special character): Jj@456
Weak password. Please follow the password criteria.
Enter a strong password (at least 8 characters, including uppercase, lowercase, digit, and special character):Joelj@456
Registration successful for user jj456.
```

```
Menu:
1. Register
2. Authenticate
3. Delete User
4. Exit
Enter your choice: 2

Enter your username: joelj
User not found. Authentication failed.

Menu:
1. Register
2. Authenticate
3. Delete User
4. Exit
Enter your choice: 2

Enter your username: joel
Enter your password: Joelj@456
Login failed. Incorrect password.

Menu:
1. Register
2. Authenticate
3. Delete User
4. Exit
Enter your choice: 2

Enter your username: joel
Enter your password: Joelj@123
Login successful! Welcome, joel.
```

```
Menu:
1. Register
2. Authenticate
3. Delete User
4. Exit
Enter your choice: 3

Enter the username to delete: joel
User joel has been deleted.

Menu:
1. Register
2. Authenticate
3. Delete User
4. Exit
Enter your choice: 2

Enter your username: joel
User not found. Authentication failed.

Menu:
1. Register
2. Authenticate
3. Delete User
4. Exit
Enter your choice: 4
Goodbye!

...Program finished with exit code 0
Press ENTER to exit console. 
```

CONCLUSION

The Multi-User Login System project provides a comprehensive solution for user authentication and management in a concurrent and secure environment. Throughout the development process, key concepts such as multi-threading, strong password enforcement, and user-friendly interfaces were incorporated to ensure robust functionality and a positive user experience.

The implementation successfully integrates multi-threading to handle concurrent user registrations, authentications, and deletions, enhancing the system's responsiveness under varying workloads. The use of mutexes ensures thread safety, preventing data conflicts and maintaining the integrity of user data.

The system enforces strong password policies, requiring users to create passwords that meet specific criteria, including length, character types, and complexity. This contributes to the overall security of the system by mitigating the risk of unauthorized access due to weak passwords.

Additionally, the project introduces a user deletion feature, allowing administrators to manage the user database effectively. This functionality enhances the system's flexibility and utility in real-world scenarios where user accounts may need to be removed.

The project's menu-driven interface simplifies user interactions, providing clear options for registration, authentication, user deletion, and system exit. User-friendly prompts and informative messages guide users through the process, contributing to an intuitive and positive user experience.

Key Findings and Contributions

Key Findings:

1. **Efficient Concurrent User Management:** The implementation of multi-threading significantly improves the system's efficiency in handling concurrent user interactions. This finding underscores the importance of concurrency management in user authentication systems, especially in scenarios with multiple simultaneous registration, authentication, and deletion requests.
2. **Enhanced Security through Strong Password Policies:** The enforcement of strong password policies contributes to heightened security levels. By requiring users to create passwords meeting specific criteria, the system minimizes the risk of unauthorized access due to weak or easily guessable passwords.
3. **Flexibility with User Deletion Feature:** The introduction of a user deletion feature adds flexibility and administrative control to the system. This feature is particularly valuable in real-world applications where user accounts may need to be removed due to various reasons, such as account closure or security concerns.
4. **User-Friendly Interface and Prompts:** The menu-driven interface and clear, user-friendly prompts contribute to an intuitive and positive user experience. This design approach is crucial for ensuring that users can easily navigate and interact with the system, promoting efficient usage.

Contributions:

1. **Multi-Threading Implementation:** The project contributes by showcasing the effective implementation of multi-threading to handle concurrent user operations. This is a valuable contribution to the field of user authentication systems, highlighting the significance of concurrency management for improved system performance.

2. Strong Password Policy Enforcement: The project emphasizes the importance of strong password policies in enhancing system security. This contribution addresses a critical aspect of user authentication, promoting the adoption of secure password practices in system design.

3. User Deletion Functionality: The inclusion of a user deletion feature adds practical value to the system, providing administrators with a tool to manage user accounts effectively. This contribution enhances the system's versatility and aligns with real-world scenarios where user accounts may need to be deactivated or removed.

4. User-Friendly Design Principles: The emphasis on user-friendly prompts and a clear menu-driven interface contributes to a positive user experience. This user-centric design approach enhances the accessibility and usability of the system, making it more approachable for users with varying levels of technical expertise.